Parallel Streams

Marco Faella
Corso di Laurea in Informatica
Corso di Linguaggi di Programmazione II
Università di Napoli Federico II

Creating Parallel Streams

- From a collection:

```
Collection::parallelStream
```

- From any other source:

```
Stream::parallel
```

They just set a “parallel” stream flag

The Last Mode Wins

```
stream.parallel()
      .filter(...)
      .sequential()
      .map(...)
      .parallel()
      .forEach(...)
```

This is a parallel pipeline

Parallel Streams

Stream elements are assigned to different threads

Each thread executes the **whole pipeline** on
a **subset of the elements**

Under the hood, this is supported by the *fork-join framework*

Basic features of the *fork-join* framework

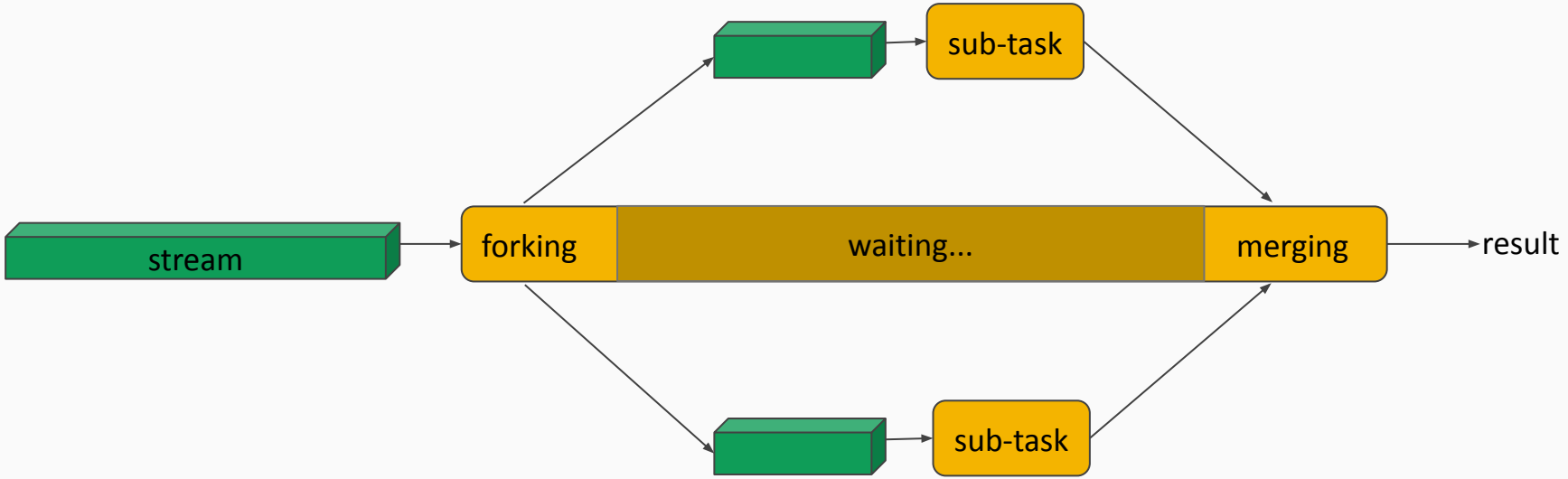
A *pool* of threads executes tasks

i.e., a limited number of threads

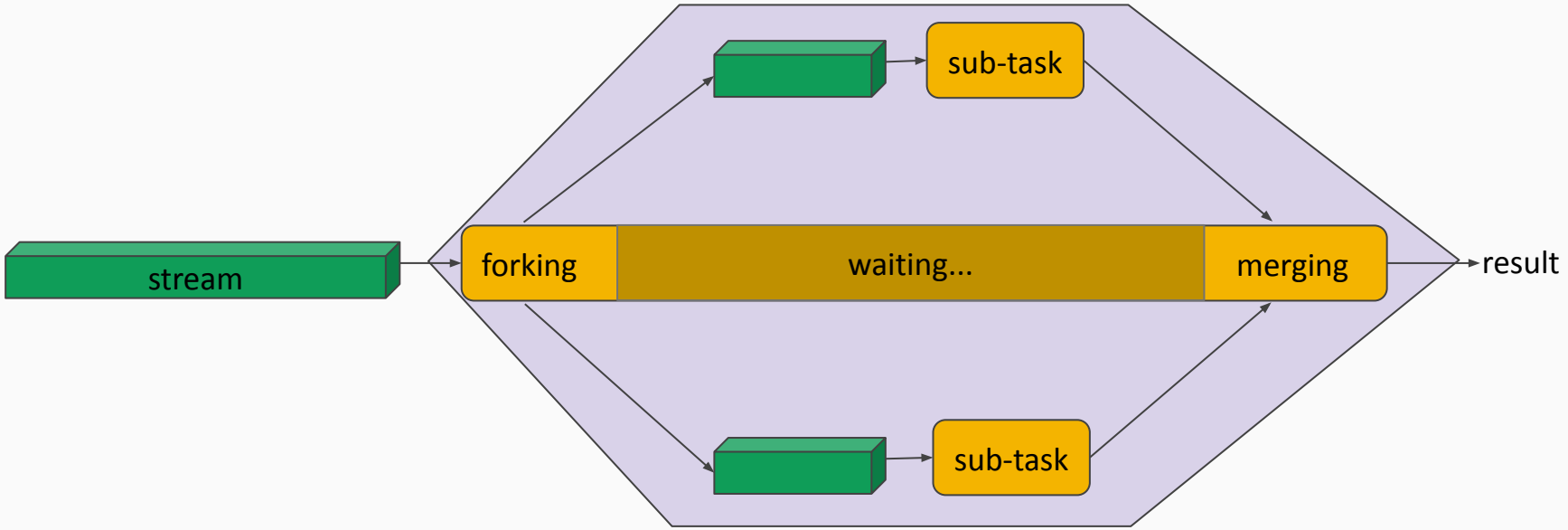
The typical task will:

- *split* (i.e., fork) into sub-tasks
- *wait* (i.e., join) for these sub-tasks to finish
- *merge* the results

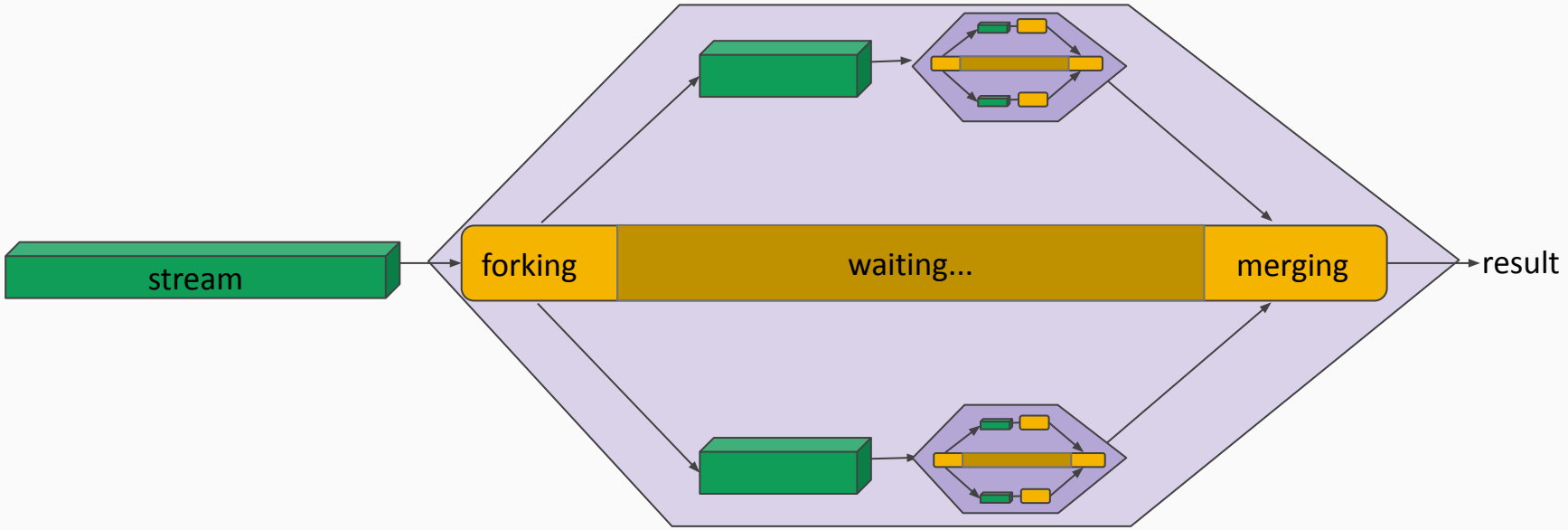
Fork-join Applied to a Stream



Fork-join Applied to a Stream



Fork-join Applied to a Stream



Recursive decomposition, up to a certain minimum size

Overheads

Splitting and merging involve substantial overhead

Parallel execution must be *worth the burden:*

- many elements, and/or
- high cost per element

Example: Compute total salary of employees

Functionally (sequential):

```
Collection<Employee> emps = ...

int totalSalary = emps.stream()
                      .mapToInt(Employee::getSalary)
                      .sum();
```

Example: Compute total salary of employees

Functionally (parallel):

```
Collection<Employee> emps = ...

int totalSalary = emps.stream()
                      .parallel()
                      .mapToInt(Employee::getSalary)
                      .sum();
```

Example: Compute total salary of employees

Imperatively (sequential):

```
class SalaryAdder {
    int total;
    public void accept(Employee e) {
        total += e.getSalary();
    }
}
SalaryAdder adder = new SalaryAdder();

emps.stream()
    .forEach(adder::accept);

int totalSalary = adder.total;
```

stateful method

accept is compatible with Consumer<Employee>

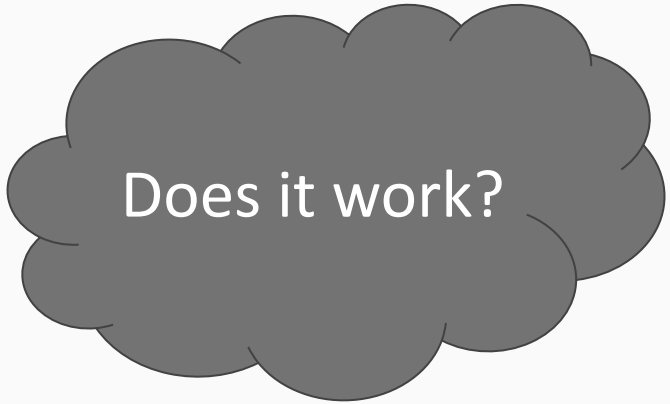
Example: Compute total salary of employees

Imperatively (parallel):

```
class SalaryAdder {
    int total;
    public void accept(Employee e) {
        total += e.getSalary();
    }
}
SalaryAdder adder = new SalaryAdder();

emps.stream()
    .parallel()
    .forEach(adder::accept);

int totalSalary = adder.total;
```



Example: Compute total salary of employees

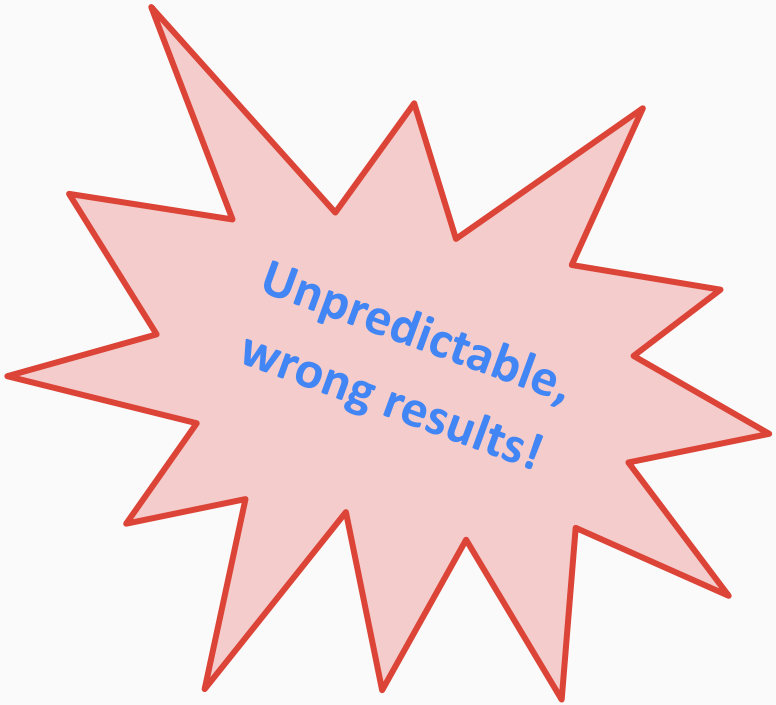
Imperatively (parallel):

```
class SalaryAdder {
    int total;
    public void accept(Employee e) {
        total += e.getSalary();
    }
}
SalaryAdder adder = new SalaryAdder();

emps.stream()
    .parallel()
    .forEach(adder::accept);

int totalSalary = adder.total;
```

race condition



Example: Compute total salary of employees

Imperatively (parallel, synchronized):

```
class SalaryAdder {
    int total;
    public synchronized void accept(Employee e) {
        total += e.getSalary();
    }
}
SalaryAdder adder = new SalaryAdder();

emps.stream()
    .parallel()
    .forEach(adder::accept);

int totalSalary = adder.total;
```

Example: Compute total salary of employees

Imperatively (parallel, synchronized):

```
class SalaryAdder {
    int total;
    public synchronized void accept(Employee e) {
        total += e.getSalary();
    }
}
SalaryAdder adder = new SalaryAdder();

emps.stream()
    .parallel()
    .forEach(adder::accept);

int totalSalary = adder.total;
```

Example: Compute total salary of employees

Imperatively (parallel, with LongAdder):

```
class SalaryAdder {
    LongAdder total;
    public void accept(Employee e) {
        total.add(e.getSalary());
    }
}
SalaryAdder adder = new SalaryAdder();

emps.stream()
    .parallel()
    .forEach(adder::accept);

long totalSalary = adder.total.longValue();
```

Example: Compute total salary of employees

Imperatively (parallel, with LongAdder):

```
class SalaryAdder {
    LongAdder total;
    public void accept(Employee e) {
        total.add(e.getSalary());
    }
}
SalaryAdder adder = new SalaryAdder();

emps.stream()
    .parallel()
    .forEach(adder::accept);

long totalSalary = adder.total.longValue();
```

Example

Computing the total salary of *50 million* employees

File `Parallel.java`

Think functionally

Pillars of Functional-Style Programming

- Immutable objects
- Stateless functions

Immutable Objects

- All final fields
- All reference fields point to immutable objects

Like String, Integer, etc.

Immutable objects

But how do I **update** an object?

You don't. You build a new one.

```
String s = "Parallel";
s += "ize";
```

s is a **new** String

Immutable objects

But how do I **update** an object?

You don't. You build a new one.

```
Employee e = new Employee("Warren", 2500);
```

Mutable Employee:

```
e.setSalary(3000);
```

Immutable Employee:

```
e = new Employee(e.getName(), 3000);
```

Stateless functions

- *Do not modify* fields of enclosing object (*this*)
- *Do not modify* static fields of enclosing class (or other classes)
- *Do not modify* the state of its arguments

Hence:

> always the same return value for the same arguments
> (a.k.a. *referential transparency*)

In addition, a *pure* function should have no side effects, so no I/O and no exceptions!

Example: List concatenation

```
MyList<T> a, b;
```

Mutable MyList, stateful concatenation:

```
a.concat(b);
```

Immutable MyList, stateless concatenation:

```
MyList<T> c = MyList.concat(a, b);
```

Parallel Summarizations

Custom Terminal Operations

Summarize a stream into a single object

Functional (i.e., *stateless*): `reduce`

Mutable (i.e., *stateful*): `collect`

Parallel `reduce`

Parallel Reduce

```
T reduce(T seed, BinaryOperator<T> accumulator)
```

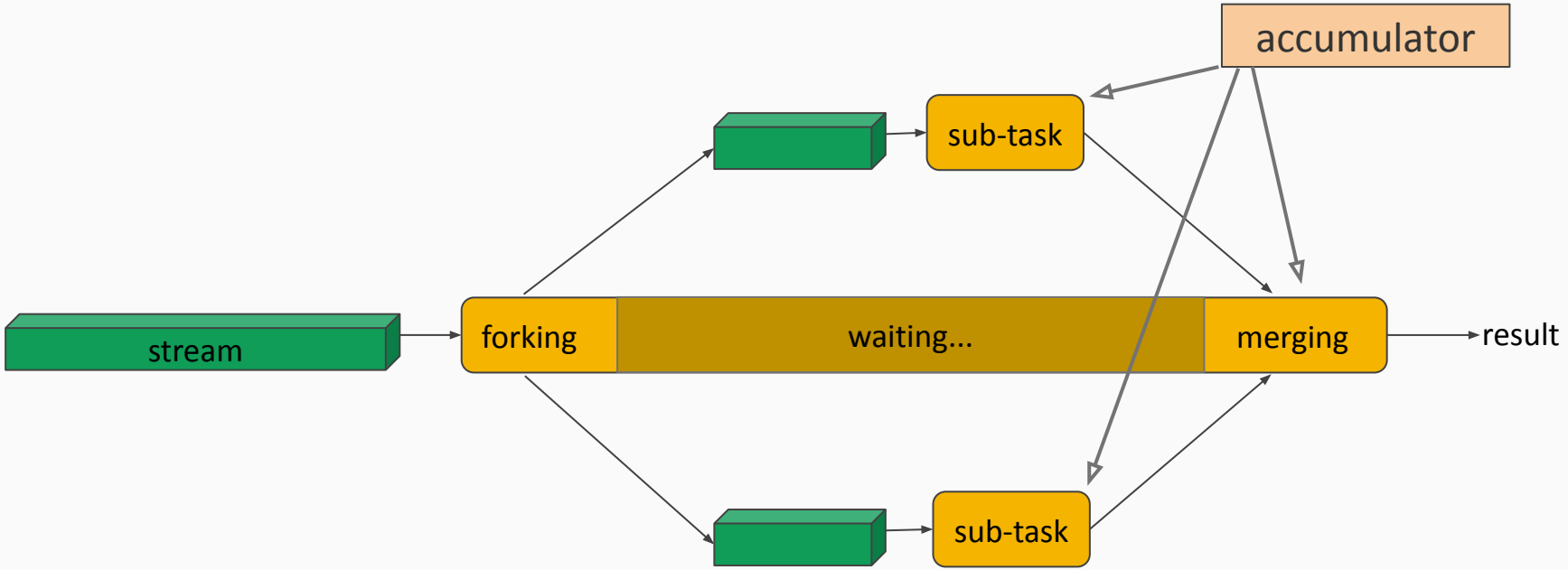
Each thread will:

- start with the same **seed**
- apply the **accumulator** to its chunk of data

When the threads finish:

- the results are merged using the **accumulator** again

Parallel Reduce in Action



Parallel Reductions

`reduce` can be efficiently parallelized, provided:

1. the binary operation (accumulator) is stateless and associative
2. the seed is an identity for the binary operation

When one of the above is false, `reduce` **will not work correctly** on parallel streams

The Binary Operation Must Be Stateless

Stateless:

```
(String a, String b) -> a + " " + b
(double a, double b) -> a * b
```

Stateful:

```
(int a, int b) -> {
                    sum += a + b;
                    if (sum>0) return a;
                    else return b;
                  }
```

where `sum` is an accessible field

The Binary Operation Must Be Associative

Order of application is irrelevant:

$$a \mathit{op} (b \mathit{op} c) = (a \mathit{op} b) \mathit{op} c$$

Associative: addition, multiplication, string concatenation

Not associative: subtraction

$$a - (b - c) = a - b + c \neq (a - b) - c = a - b - c$$

The Seed Must be an Identity

It does not affect the result of the operation:

seed *op* a = a

Examples:

- if *op* is +, seed must be 0
- if *op* is *, seed must be 1
- if *op* is String::concat, seed must be “”

Parallel `collect`

Parallel Collect

In `Stream<T>`:

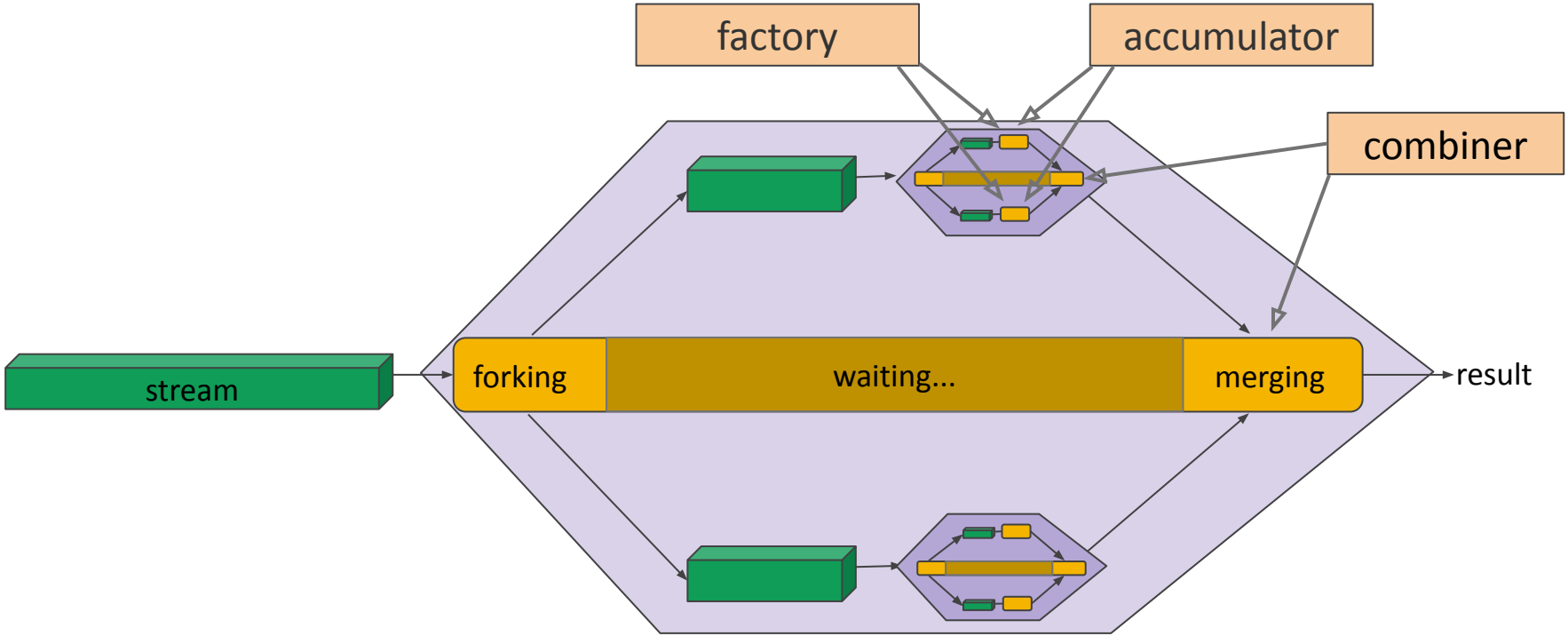
```
<S> S collect(Supplier<S>     factory,
              BiConsumer<S,T> accumulator,
              BiConsumer<S,S> combiner)
```

Each thread will:

- invoke the **factory** to get an initial summary
- update the summary by applying the **accumulator** to its chunk of data

When the threads finish:

- the results are merged using the **combiner**



Factory, accumulator, and combiner are invoked concurrently by multiple threads!

Parallel Collectors

`collect` can be efficiently parallelized, provided:

1. the factory is stateless
2. the accumulator is stateless and associative
3. the combiner is stateless and associative

When one of the above is false, `collect` **may not work correctly** on parallel streams

Isn’t a stateless accumulator a contradiction?

Stateless accumulator?

- The summary is **mutable**
- The accumulator and the combiner should be **stateless**

- The accumulator and the combiner…
  do not return any value,
  modify (one of) their arguments,
  (which is a **side-effect)**
  but should be **stateless**

In other words:

given the same inputs, they should modify the summary in the same way

Other Parallel Operations

Know Your Pipeline

Operations have (very) different parallel attitudes:

- Some parallelize *gracefully*
- Others *resist* parallelization

Compared to sequential,
performance can **improve** or **deteriorate**!

Know your operations

Build operations:

How easy is it to **split** the stream?

Intermediate and **terminal** operations:

Is the operation performed on each element **separately**?

How easy is it to **merge** the (partial) results?

Splitting a Stream

Parallel streams repeatedly split stream into halves

The build operation must support **jumping to the middle element**

(that's where the second half starts from)

Splitting a stream built from a collection

- plain array: easy
- ArrayList: easy

- HashSet: medium
- TreeSet: medium

- LinkedList: hard must traverse first half

Prefer Arrays and ArrayLists

In some cases, it may be useful to
first transfer all elements into an array

Splitting a stream built from a computation

• **generate**	easy	each element is generated separately
• IntStream::**range**	easy	regular and known in advance
• **iterate**	hard	each element generated from previous

Parallel-friendly intermediate operations

- `filter`
- `map`

- They work on each element separately*
- Partial results need only be concatenated

* Provided the functional argument is stateless

Parallel-unfriendly intermediate operations (2)

- `distinct`
- `sort`

- *Can* process different chunks of elements separately
- Merging requires **partial re-processing**

`distinct` can be sped-up by *unordering* the stream

Standard terminal operations

Most are parallel-friendly

(provided the functional arguments are stateless)

`forEach`, `count`,
`allMatch`, `anyMatch`,
`findAny`, `findFirst`,
`reduce`
`sum`, `max`, `min`

Collectors

Standard collectors are parallel-friendly

`toList`, `toSet`, etc.

Grouping collectors are relatively efficient

`toMap`, `groupingBy`, etc.

If order is not relevant, use their *concurrent* versions

`toConcurrentMap`, `groupingByConcurrent`, etc.

Custom collectors

Custom mutable summarizations (`collect` method):

- sequential: parallel performance depends on the *combiner*
- concurrent: parallel performance depends on the *accumulator*

2D Collision Detection

A Parallel Stream Exercise

A mini-project with streams

- Bounding shapes and bounding boxes
- Basic 2D collision detection
- Using streams to parallelize
- Performance tests

2D Games

- Game objects move on screen and possibly collide
- Some collisions give rise to game events

- Problem: Efficient collision detection

Bounding shapes

- Game objects are depicted as **sprites** (bitmap images)
- Sprites have *arbitrary contours*
- Exact collision detection is too time-consuming

Replace sprites with simpler *bounding shapes*

Simple bounding shapes

Bounding circle

Axis-aligned bounding box (AABB)

Performance optimization

We need to check **all pairs** of objects: **quadratic** complexity
Optimization idea: use two phases

1. **Broad phase:**
   We split the scene in **regions**, and group the objects accordingly
   → linear time

2. **Narrow phase:**
   We check collisions **within each region** (objects in different regions cannot collide)
   → quadratic time, *on fewer pairs*

Grouping in a grid

Example

Assume **100** objects in the scene

With a single phase phase:

a collision test for each **unordered pair of distinct objects**

100*99/2 = **4.950 collision tests**

With two phases (broad/narrow):

Let's split the scene in two regions, with 50 objects each

In each region: 50*49/2 = **1.225 collision tests**

Total: **2.450** collision tests

50% savings

Implementation (single phase)

Files:

`Box.java`	an axis-aligned bounding box
`Collision.java`	a collision between two boxes
`BasicDetection.java`	one-phase detection algorithms
`ForEachCollisionCollector.java`	a custom collector
`CountingTest.java`	collision counting tests

Implementing Broad and Narrow Phase

Files:

`GridSpec.java`	grid parameters
`GridCollector.java`	*broad phase:* groups boxes in grid cells
`CollisionListCollector.java`	*narrow phase:* collects collisions in a list
`GridDetection.java`	two-phase grid-based algorithms
`AllCollisionsTest.java`	collision collecting tests

Two-Phase Implementation: Summary

- Two-phase algorithm may improve the performance
- Subtle performance trade-offs require experimentation

How “Functional” is this?

- *Functional-style*, not *functional*
- We exploit functional-style streams
- We pass self-contained behavior (such as collectors) to streams
- The collectors themselves are mutable
- This compromise is the most efficient way, in Java